EEET2601 Engineering Computing 1

More Data Types & Operators

Quang Tran

RMIT UNIVERSITY

User-define data types

- So far, we have used the primitive data types (char, short, int, long, float, and double)
- What if the primitive data types do not suit your needs?
  - Example: you need to work with thousands of student records. Each record contains many details such as id, name, date of birth, phone number, address, etc.
- C allows you to create and use your own data types

Structure

- An **array** is a collection of elements of a **same data type**
- A **structure** is a collection of elements of **different data types**
- Array and structure are often used together to create a database for a C program

↳ a set of data

Structure declaration

```
struct struct_name {
    data_type1 var1;
    data_type2 var2;
    ...
};
struct struct_name var;
```

↘ varible of struct

Example: Declare and use structure

```c
#include <stdio.h>

struct student {
    int id;
    char name[30];
    int age;
};

void print_student(struct student stu) {
    printf("%d %s %d\n", stu.id, stu.name, stu.age);
}

int main() {
    struct student paul = {12345, "Paul Smith", 22};
    print_student(paul);
    return 0;
}
```

Define a structure named **student**

id, name and age are **members** of struct student

Access members of std using the **“.” operator**

Declare and initialize a variable **paul** of type struct student

Pass **paul** to print_student

cannot change the value, double mem usage

Example: Pointer to structure

```c
#include <stdio.h>

struct student {
    int id;
    char name[30];
    int age;
};

void print_student(struct student *p) {
    printf("%d %s %d\n", p->id, p->name, p->age);
}

int main() {
    struct student paul = {12345, "Paul Smith", 22};
    print_student(&paul);
    return 0;
}
```

*p.id

Define a structure named **student**

id, name and age are **members** of struct student

Access members of of a struct through a pointer using the **"->" operator**

Declare and initialize a variable **paul** of type struct student

Pass **&paul** to the function

Example: Array of structure

```c
#include <stdio.h>
#include <string.h>

struct student {
    int id;
    char name[30];
    int age;
};

void print_students(struct student students[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d %s %d\n", students[i].id, students[i].name, students
        [i].age);
    }
}

int main() {
    const int SIZE = 2;
    struct student records[SIZE];

    records[0].id = 1;
    strcpy(records[0].name, "John Smith");
    records[0].age = 20;

    records[1].id = 2;
    strcpy(records[1].name, "Alice Singleton");
    records[1].age = 19;

    print_students(records, SIZE);

    return 0;
}
```

name = "..." ×
can't assign to pointer

Define a function to print an array of students

Define a constant integer for array size

Declare an array of students

Assign values to each student

Call function to print students

typedef

- **typedef** allows to create an alias of a data type
- Syntax: `typedef data_type new_data_type_name;`
- Example: `typedef char byte;`
- **typedef** is often used with **struct** to create a short data type

Example: typedef and struct

```c
#include <stdio.h>

typedef unsigned char byte;

typedef struct {
    int id;
    char name[30];
    byte age;
} student;

void print_student(student stu) {
    printf("%d %s %d\n", stu.id, stu.name, stu.age);
}

int main() {
    student paul = {12345, "Paul Smith", 22};
    print_student(paul);
    return 0;
}
```

Create a new data type named **byte**

Use typedef and struct together to create a new data type named **student**

The new data type **byte** and **student** can be used like other basic data types

enum

- **enum** (abbreviation of enumeration) is used to assign names to **integer constants** to make a program easier to read and maintain
- Syntax: `enum enum_name {CONST1, CONST2, …};`
- Examples:
  - `enum boolean {FALSE, TRUE};`
  - `enum day {MON, TUE, WED, THU, FRI, SAT, SUN};`

enum

- By default, C assigns 0, 1, 2, … to the constants in the enum declaration.
- However, you can assign other integer values explicitly if you wish.
- Example: `enum day {MON = 1, TUE = 2, WED = 3, THU = 4, FRI = 5, SAT = 6, SUN = 7};`

Example: enum

```c
#include <stdio.h>

enum day {MON = 1, TUE, WED, THU, FRI, SAT, SUN};

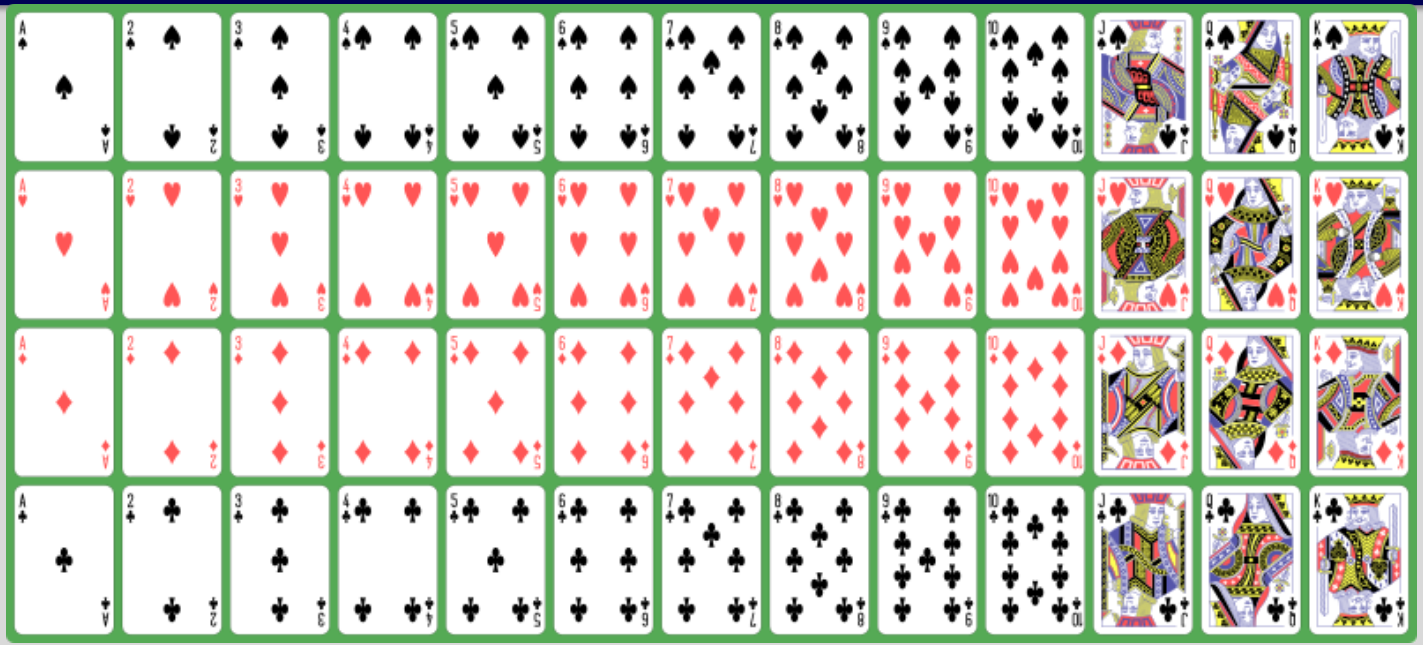
int main() {
    for (enum day myday = MON; myday <= SUN; myday++)
        printf("%d ", myday);
    printf("\n");

    return 0;
}
```

Example: A deck of cards

- Write a program to shuffle a standard deck of 52 cards then print the cards
- Steps:
  1. Analysis:
     - What are the cards in a standard deck of 52 cards?
     - What info does a card have?
  2. Design: Design the data types, the program structure, and key algorithms
     - How to represent a card and a deck of 52 cards?
     - How to shuffle the cards? How to print the cards?
  3. Coding: Implement the program
  4. Testing: Compile, run and test the program

Example: A deck of cards



References

1. Fresh2Refresh, C Programming Tutorial, 2021.
2. StudyTonight, C Programming Language Tutorial, 2021.
3. Wikipedia, Standard 52-card deck, 2021.